developer.nvidia.com

# Run Multimodal Extraction for More Efficient AI Pipelines Using One GPU

*Lior Cohen*

9–11 minutes

---

As enterprises generate and consume increasing volumes of diverse data, extracting insights from multimodal documents, like PDFs and presentations, has become a major challenge. Traditional text-only extraction and basic retrieval-augmented generation (RAG) pipelines fall short, failing to capture the full value of these complex documents. The result? Missed insights, inefficient workflows, and rising operational costs.

In this blog post, we'll dive into the key components of building an effective multimodal extraction pipeline, with code examples and one GPU, using NVIDIA NeMo Retriever microservices.

NeMo Retriever extraction is an example architecture for a multimodal document processing pipeline. It uses microservices to efficiently extract information from different file types across millions of documents. Paired with NeMo Retriever embedding and reranking models, it forms a complete, scalable RAG solution as showcased in NVIDIA AI Blueprint for RAG (described in Figure 1).

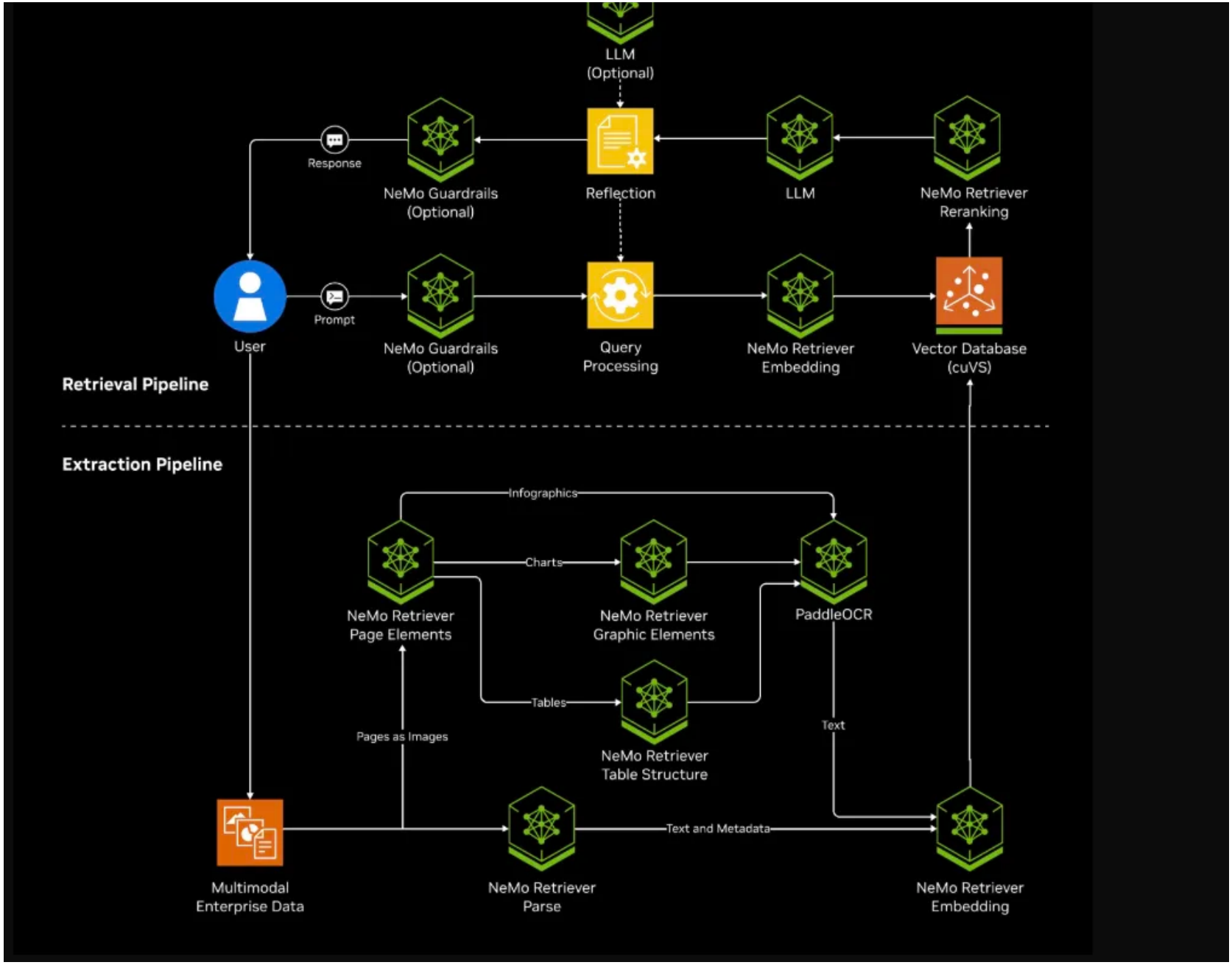


*Figure 1. A diagram of the extraction pipeline as part of the Build an Enterprise RAG NVIDIA AI Blueprint*

In the next section, we'll take a simple use case that shows the step-by-step NeMo Retriever extraction pipeline, together with other NeMo Retriever components, to use the ingested data.

## Completing a business task with multimodal document understanding

For this purpose, we'll consider an organizational shared folder containing product resources about NVIDIA Blackwell GPUs. The files include text, images, charts, and tables in different types such as PDFs, PPTXs, and JPEGs.

In this example, a customer support engineer is asking for an NVIDIA Blackwell performance comparison to support a partner.

This type of request can be sent by any application, such as a chat user interface or an automatic content generator. Here, we'll demonstrate it using direct prompting with the pipeline's Python client.

### Step 1: Spin up the pipeline with one GPU

Deploy the blueprint using Docker on an on-prem or cloud machine. See the deployment guide available in the [NeMo Retriever extraction quickstart guide on NVIDIA docs](#).

In this example, the entire pipeline was deployed on an AWS g6e.xlarge machine (L40S GPU, 48 GB). Verify all deployed services for your desired profile are up and running.

```
(end-to-end-rag-pipeline) ubuntu@i[redacted]:~$ docker ps --format "table {{.Image}}\t{{.Names}}"
IMAGE                                                    NAMES
zilliz/attu:v2.3.5                                       milvus-attu
milvusdb/milvus:v2.5.3-gpu                               milvus-standalone
otel/opentelemetry-collector-contrib:0.91.0              nv-ingest-otel-collector-1
nvcr.io/nim/baidu/paddleocr:1.2.0                        nv-ingest-paddle-1
nvcr.io/nim/nvidia/nemoretriever-graphic-elements-v1:1.2.0   nv-ingest-graphic-elements-1
nvcr.io/nim/nvidia/nemoretriever-page-elements-v2:1.2.0      nv-ingest-page-elements-1
redis/redis-stack                                        nv-ingest-redis-1
nvcr.io/nim/nvidia/llama-3.2-nv-embedqa-1b-v2:1.5.0      nv-ingest-embedding-1
quay.io/coreos/etcd:v3.5.5                               milvus-etcd
grafana/grafana                                          grafana-service
prom/prometheus:latest                                   nv-ingest-prometheus-1
nvcr.io/nim/nvidia/nemoretriever-table-structure-v1:1.2.0    nv-ingest-table-structure-1
```

```
nvcr.io/nvidia/nemo-microservices/nv-ingest:25.3.0          nv-ingest-nv-ingest-ms-runtime-1
openzipkin/zipkin                                           nv-ingest-zipkin-1
minio/minio:RELEASE.2023-03-20T20-16-18Z                    minio
```

*Figure 2. NVIDIA NeMo Retriever extraction deployed services*

The pipeline services include the visual elements recognition and OCR (optical character recognition) models, the embedding model, Milvus DB, and the observability tools (Prometheus and Grafana, Attu, Zipkin, and more).

Note: for prototyping purposes, the pipeline source code can be accessed at the blueprint page, [Build an Enterprise RAG pipeline](#).

### Step 2: Submit ingestion job for files in storage

Once all services are up, we can submit ingestion jobs either by Python client or through CLI (command line interface).

We'll show the Python client usage.

In this example, we pass the path for the files in our collection and define the tasks we want to be included in the job (i.e., extract, split, and embed). We set the extraction task to include all modalities types, and the split task to chunk text to a size of 1,024 tokens.

```
from nv_ingest_client.client import Ingestor

demo_files ="demo_files/*"

ingestor =(

    Ingestor(message_client_hostname="localhost")

    .files(demo_files)
```

```
    .extract(
        extract_text=True,
        extract_tables=True,
        extract_charts=True,
        extract_images=True,
        text_depth="page",
    )
    .dedup()
    .split(
        tokenizer="meta-llama/Llama-3.2-1B",
        chunk_size=1024,
    )
    .embed()
    .vdb_upload()
)
result = ingestor.ingest()
```

**Step 3: Analyze the job results**

Once the ingestion job is completed, we can analyze the structure of the results (Figure 3):

```
import pandas as pd
df = pd.DataFrame([])
for doc in result:
    for obj in doc:
        df = pd.concat([df,pd.json_normalize(obj)])
display(df)
```

| | document_type | metadata.content | metadata.content_url | metadata.embedding | metadata.source_metadata.sour |
|---|---|---|---|---|---|
| 0 | text | NVIDIA GB200 NVL72 \| Datasheet \| 1\r\nNVIDIA G... | | [9.804964065551758e-05, -0.011825561523437 5, -... | demo_files/blackwell_ |
| 0 | text | NVIDIA GB200 NVL72 \| Datasheet \| 2\r\nMassive-... | | [0.006458282470703125, -0.010711669921875, -0.... | demo_files/blackwell_ |
| 0 | text | NVIDIA GB200 NVL72 \| Datasheet \| 3\r\nData Pro... | | [-0.007595062255859375, 0.0219573974609375, -0... | demo_files/blackwell_ |
| 0 | text | Ready to Get Started?\r\nTo learn more about t... | | [-0.0022754669189453125, 0.01232147216796875, ... | demo_files/blackwell_ |
| 0 | structured | iVBORw0KGgoAAAANSUhEUgAAAXcAAAEcCAIAAACzkWMwAA... | | [0.0103759765625, -0.023895263671875, -0.02630... | demo_files/blackwell_ |
| 0 | structured | iVBORw0KGgoAAAANSUhEUgAAAWkAAAEICAIAAACpg4QOAA... | | [0.002838134765625, -0.01409912109375, -0.0111... | demo_files/blackwell_ |
| 0 | structured | iVBORw0KGgoAAAANSUhEUgAAAXoAAAEbCAIAAABbatg4AA... | | [-0.0115509033203125, -0.0187225341796875, -0.... | demo_files/blackwell_ |
| 0 | structured | iVBORw0KGgoAAAANSUhEUgAAAWwAAAEoCAIAAADzNFyUAA... | | [-0.01116943359375, -0.007419586181640625, -0.... | demo_files/blackwell_ |
| 0 | structured | iVBORw0KGgoAAAANSUhEUgAAArkAAAIaCAIAAACrtMKjAA... | | [-0.00882720947265625, -0.00937652587890625, 0... | demo_files/blackwell_ |
| 0 | image | iVBORw0KGgoAAAANSUhEUgAAAOAAAADfCAIAAABgY0jOAA... | | None | demo_files/blackwell_ |
| 0 | image | iVBORw0KGgoAAAANSUhEUgAAAIUAAACHCAIAAAC3cA1gAA... | | None | demo_files/blackwell_ |
| 0 | image | iVBORw0KGgoAAAANSUhEUgAAAP8AAAByCAIAAAATRJCbAA... | | None | demo_files/blackwell_ |
| 0 | image | iVBORw0KGgoAAAANSUhEUgAAAP8AAABxCAIAAACV0OI1AA... | | None | demo_files/blackwell_ |

*Figure 3. Extraction job results*

The job extraction resulted in several objects from different modalities for each of our documents. These include text, images, and structured objects that refer to charts and tables.

For example, we can see a text object that was extracted:

```
# print a random text object that was extracted.
print(df[df['document_type']=='text'].sample(1)['metadata.content'][0])
# Output:
# NVIDIA GB200 NVL72 | Datasheet | 1
# NVIDIA GB200 NVL72
# Powering the new era of computing.
# Unlocking Real-Time Trillion-Parameter Models
# NVIDIA GB200 NVL72 connects 36 Grace CPUs and 72 Blackwell GPUs in an NVIDIA®
# NVLink®-connected, liquid-cooled, rack-scale design. Acting as a single, massive GPU, it
# delivers 30X faster real-time trillion-parameter large language model (LLM) inference.
# The GB200 Grace Blackwell Superchip is a key component of the NVIDIA GB200
# NVL72, connecting two high-performance NVIDIA Blackwell GPUs and an NVIDIA...
```

The text can also be split into smaller chunks. We can control the chunking strategy in our split configuration of the Ingestor.

This is an example of  a random table that was extracted:

```
# detected chart
from base64 import b64decode
from IPython import display
rand_extracted_object = df[df['document_type']=='structured'].sample(1)
display.Image(b64decode(rand_extracted_object['metadata.content'][0]))
```

## Product Specifications[1]

The NVIDIA GB200 Grace Blackwell Superchip comes in two configurations: GB200 NVL72 and GB200 NVL2.

| Feature | GB200 NVL72 | GB200 NVL2 | GB200 Grace Blackwell Superchip |
|---|---|---|---|
| **Configuration** | 36 Grace CPUs, 72 Blackwell GPUs | 2 Grace CPUs, 2 Blackwell GPUs | 1 Grace CPU, 2 Blackwell GPUs |
| **FP4 Tensor Core[2]** | 1,440 PFLOPS | 40 PFLOPS | 40 PFLOPS |
| **FP8/FP6 Tensor Core[2]** | 720 PFLOPS | 20 PFLOPS | 20 PFLOPS |
| **INT8 Tensor Core[2]** | 720 POPS | 20 POPS | 20 POPS |
| **FP16/BF16 Tensor Core[2]** | 360 PFLOPS | 10 PFLOPS | 10 PFLOPS |
| **TF32 Tensor Core[2]** | 180 TFLOPS | 5 PFLOPS | 5 PFLOPS |
| **FP32** | 6,480 TFLOPS | 180 TFLOPS | 180 TFLOPS |
| **FP64** | 3,240 TFLOPS | 90 TFLOPS | 90 TFLOPS |
| **FP64 Tensor Core** | 3,240 TFLOPS | 90 TFLOPS | 90 TFLOPS |
| **GPU Memory \| Bandwidth** | Up to 13.5TB HBM3e \| 576TB/s | Up to 384GB HBM3e \| 16TB/s | Up to 384GB HBM3e \| 16TB/s |
| **NVLink Bandwidth** | 130TB/s | 3.6 TB/s | 3.6TB/s |
| **CPU Core Count** | 2,592 Arm Neoverse V2 cores | 144 Arm Newoverse V2 cores | 72 Arm Neoverse V2 cores |
| **CPU Memory \| Bandwidth** | Up to 17TB LPDDR5X \| Up | Up to 960GB LPDDR5X \| Up | Up to 480GB LPDDR5X \| Up |

| | to 18.4TB/s | to 1,024GB/s | to 512GB/s |
|---|---|---|---|
| **Form Factor** | MGX Rack | MGX | Module |

*Figure 4. A table extracted by the ingestion job*

In addition to the visual object extraction, the textual content of it is saved as well:

```
rand_extracted_object['metadata.table_metadata.table_content'][0]
```

These objects were chunked and embedded automatically by the job. We can track the vector embeddings in the Milvus collection created automatically by the pipeline through the Milvus client or Attu (web user interface for Milvus) service that was deployed with the rest of the services bundle in step 1.

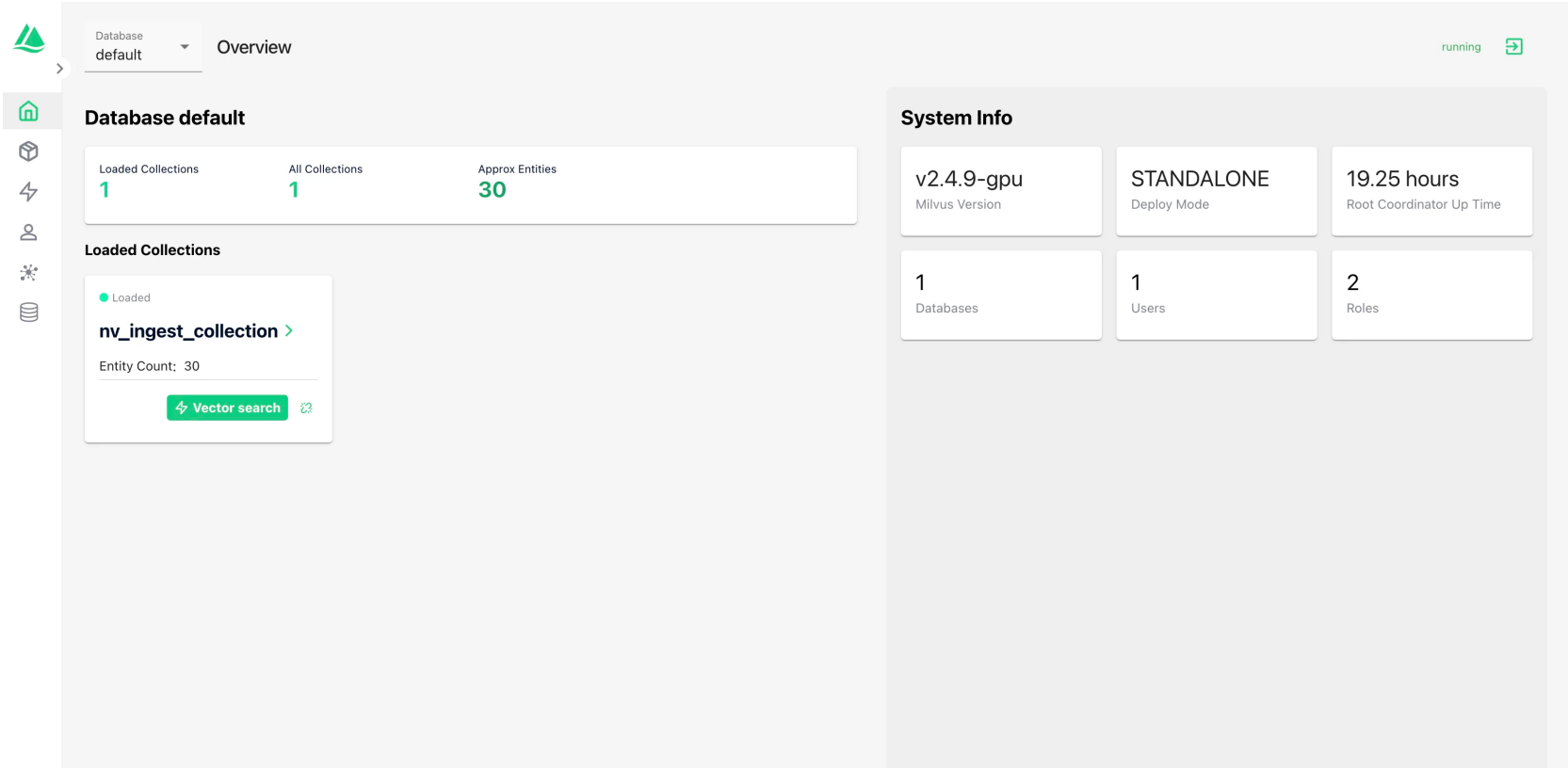

*Figure 5. The Milvus collection was automatically created by the NVIDIA NeMo Retriever extraction pipeline*

### Step 4: Retrieval

We'll demonstrate building a retrieval component based on the NeMo Retriever extraction pipeline, ingested data, and NeMo Retriever embedding. First, define the NVIDIA client for the embedding and the generator LLM microservices.

Embed the user query (using the same embedding model used in the ingestion)

```
user_query ="I am a customer support engineering asking for my client –
What is the main difference between the two configurations of grace
blackwell?"

response =nvidia_client.embeddings.create(

    input=user_query,

    model="nvidia/nv-embedqa-e5-v5",

    encoding_format="float",

    extra_body={"input_type": "query", "truncate": "NONE"}

)

user_query_vector=response.data[0].embedding
```

Get top similar results to the user's query using NeMo Retriever extraction Python client retriever:

```
from nv_ingest_client.util.milvus import nvingest_retrieval

query_results = nvingest_retrieval(
```

```
    [user_query],
    "nv_ingest_collection",
    hybrid=False,
    model_name="nvidia/llama-3.2-nv-embedqa-1b-v2",
    top_k=1,
    gpu_search=True,
)
top_result = query_results[0][0]['entity']['text']
```

Create a relevant prompt for the generator LLM and get the response:

```
prompt =
.format(top_result, user_query)
completion =nvidia_client.chat.completions.create(
  model="meta/llama-3.2-3b-instruct",
  messages=[{"role":"user","content":prompt}],
  temperature=0.2,
  top_p=0.7,
  max_tokens=200,
  stream=True
```

```
)
for chunk in completion:
  if chunk.choices[0].delta.content is not None:
    print(chunk.choices[0].delta.content, end="")
```

We retrieved a highly relevant chunk that didn't require any direct search and review of the original file.

This simple use case shows how, with a quick deployment setup, we can perform automatic contextual understanding of multimodal enterprise source files.

## Conclusion

The NeMo Retriever extraction pipeline addresses the challenges of multimodal document processing by automatically handling different file types, such as PDFs, presentations, and spreadsheets.t extracts meaningful content from text, images, tables, and charts, changing previously siloed information into accessible, structured data. This enables organizations to unlock deeper insights from their existing knowledge repositories.

The architecture behind this solution brings together advanced components like object detection, chart parsing, and vector embeddings to enable efficient, context-aware retrieval. By preserving relationships across modalities and surfacing them through semantic search, the pipeline delivers a comprehensive approach to document understanding. Implementing this end-to-end pipeline with NeMo Retriever marks a major advancement in enterprise knowledge management, turning static, underutilized documents into high-value assets that can fuel generative AI applications and smarter decision-making.

By continuously extracting and using new data, NeMo Retriever can also help organizations create a

data flywheel, where improved data quality leads to better AI models, which in turn generate even more valuable data.

Get started with the NeMo Retriever extraction pipeline using the NVIDIA AI blueprint for RAG, or try the individual NeMo Retriever microservices for extraction, embedding, and reranking on build.nvidia.com.